

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Lovro Alfirević

Josip Ančić

Detekcija phishing poruka (email/SMS)
klasičnim ML-om

PROJEKTNNA DOKUMENTACIJA

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Lovro Alfirević

Josip Ančić

Studij: Informacijski sustavi

Studij: Informacijski sustavi

Detekcija phishing poruka (email/SMS)
klasičnim ML-om

PROJEKTNA DOKUMENTACIJA

Mentorica:

Izv. prof. dr. sc. Dijana Oreški

Varaždin, siječanj 2026.

Lovro Alfirević i Josip Ančić

Izjava o izvornosti

Izjavljujemo da je naš projektni rad izvorni rezultat našeg rada te da se u izradi istoga nismo koristili drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autori su potvrdili prihvaćanjem odredbi u sustavu ELF FOI (Moodle)

Sažetak

Tema ovog rada je razvoj sustava za automatsku detekciju zlonamjernih (*phishing*) poruka unutar elektroničke komunikacije (e-mail i SMS). S obzirom na rastući broj kibernetičkih napada, cilj je bio izraditi algoritam koji može s visokom točnošću razlikovati legitimne poruke od onih koje pokušavaju prevariti korisnika. U radu su primijenjene metode strojnog učenja za analizu teksta. Praktični dio rada uključuje implementaciju klasifikatora te izradu aplikacije koja demonstrira uspješnost prepoznavanja neželjene pošte u stvarnom vremenu.

Sadržaj

1. Uvod	1
2. Metode i tehnike rada	2
2.1. Prikaz konverzacije sa AI-alatom	2
3. Razrada teme	6
3.1. Phishing i spam.....	6
3.2. Metode strojnog učenja.....	7
3.1.1. Multinomial Naive Bayes (NB).....	7
3.1.1. Logistička regresija (LogReg) eće razine.....	7
3.1.1. Support Vector Machine (SVM) treće razine	7
3.3. Kritički osvrt	8
3.4. Opis Implemetacije.....	9
3.4.1. data_prep.py	9
3.4.1. pomocne_klase_modela.py.....	10
3.4.1. train.py	12
3.4.1. guiTkinter.py.....	13
3.5. Prikaz rada aplikacije	16
4. Zaključak	20
Popis literature	21
Popis slika	21

1. Uvod

Elektronička pošta (e-mail) i SMS poruke danas predstavljaju primarnu vrstu poslovne i privatne komunikacije. Porastom volumena digitalne komunikacije, proporcionalno raste i broj zlonamjernih aktivnosti, među kojima se ističe *phishing*. „This attack method, commonly known as “phishing” is most commonly initiated by sending out emails with links to spoofed websites that harvest information [1].“Motivacija za ovaj projekt proizlazi iz potrebe za automatiziranim i efikasnim sustavima detekcije koji mogu filtrirati spam i zlonamjerne poruke. „By receiving spam messages, Internet users are exposed to security issues, and minors are exposed to illegal and improper contents [2].“ Uz to što spam poruke mogu biti zlonamjerne, one također troše važne resurse poput pohrane, propusnosti i produktivnosti (Faris, Al-Zoubi, Heidari, Aljarah, Mafarja, Hassonah, i Fujita, 2019.).

Cilj rada je razviti i dokumentirati sustav koji kombinira analizu teksta na leksičkoj razini koja može prepoznati je li poruka zlonamjernog karaktera ili ne.

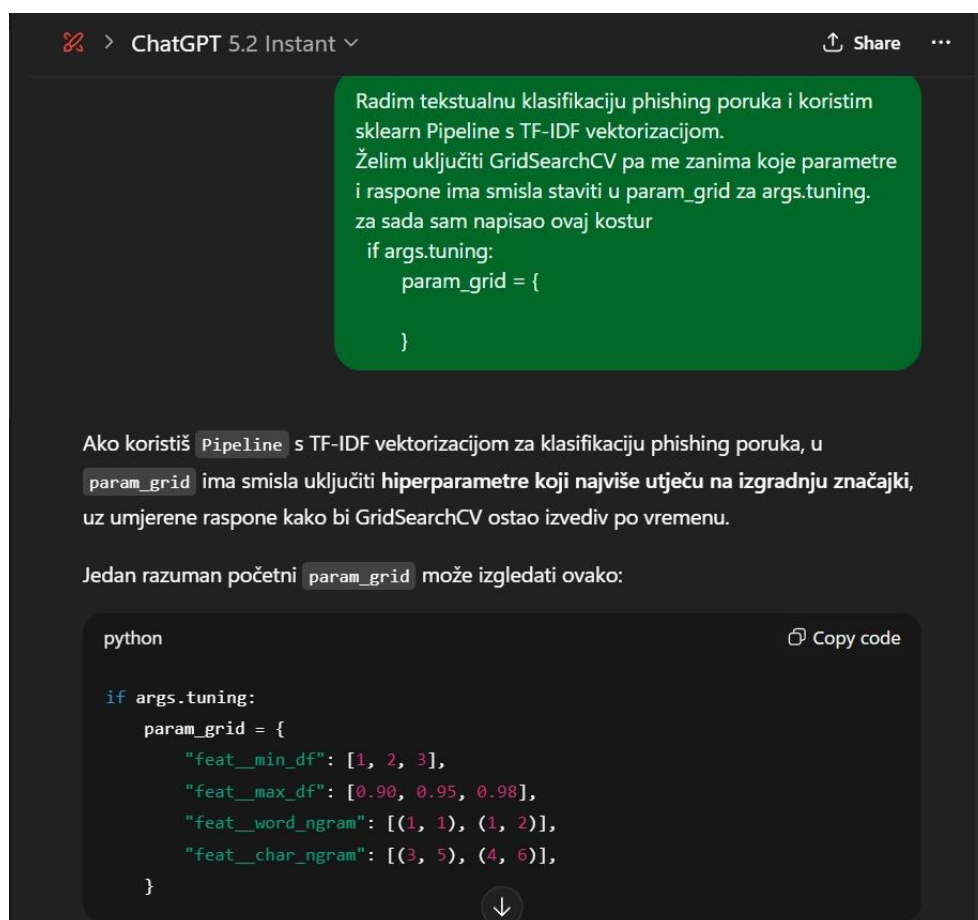
2. Metode i tehnike rada

U našem projektu za svrhe stvaranja koda smo koristili virtualno okruženje Visual Studio Code, u kojem smo naše korisničko sučelje i cijeli algoritam koji detektira phishing email-ove i SMS-ove kodirali u programskom jeziku Python. U procesu pisanja koda imali smo nekih problema koje nismo znali sami riješiti, stoga smo se za taj problem poslužili umjetnom inteligencijom (više o samom razgovoru i korištenju umjetne inteligencije nalazi se u sljedećem potpoglavlju). Za pisanje projektne dokumentacije koristio se markup jezik LaTeX. Od literature koristili smo nekoliko znanstvenih članaka, izvora sa web provjerenih mjesta te prezentacija od pripadajućeg kolegija. Također unutar metoda rada smo se koristili materijalima sa laboratorijskih vježbi.

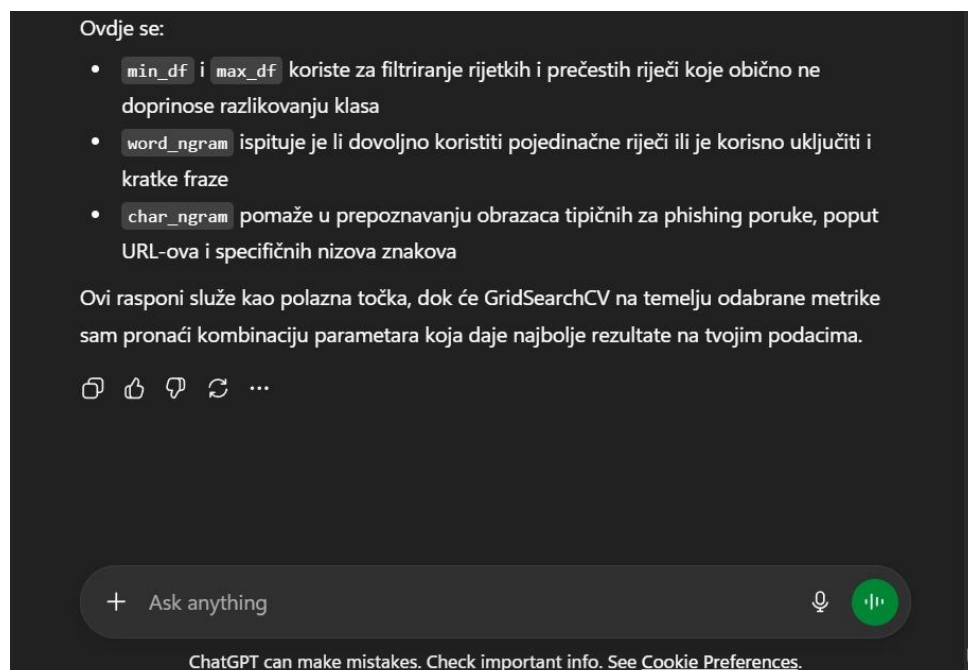
2.1. Prikaz konverzacije sa AI-alatom

U našem projektu smo koristili AI alat naziva ChatGPT, to je standardni LLM (eng. Large Language Model) globalno poznat. Problem sa kojim smo se suočili je uključivao tekstualnu klasifikaciju phishing poruka. Pošto smo htjeli uključiti GridSearchCV zanimalo nas je koje parametre i raspone bismo trebali staviti u kod da bi on kvalitetno radio. Također smo tražili od umjetne inteligencije da nam kaže koji parametri bi bili najlogičniji da se implementiraju u param_grid za modele NB, logističke regresije i metode potpornih vektora.

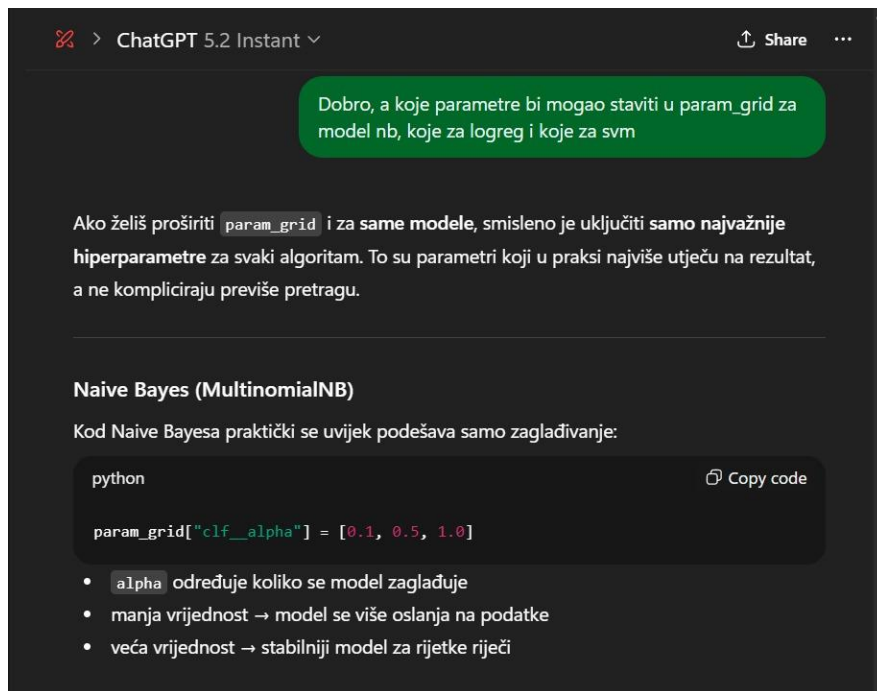
U sljedećim fotografijama je prikazan naš razgovor sa AI alatom ChatGPT.



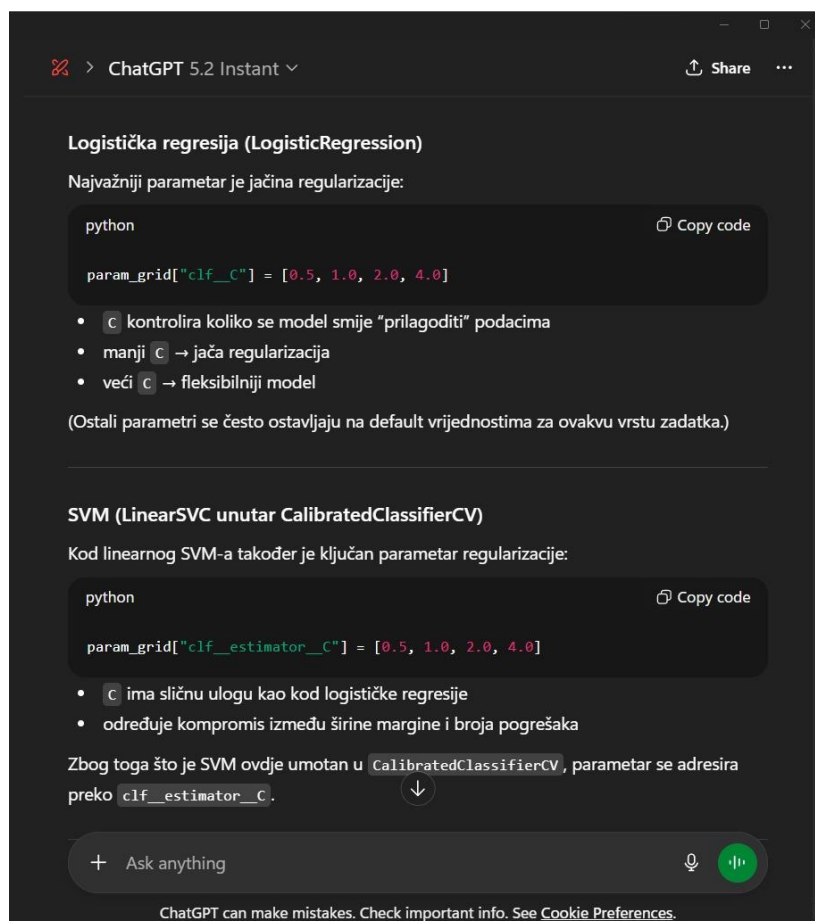
Konverzacija AI alat 1/5 (autorski rad)



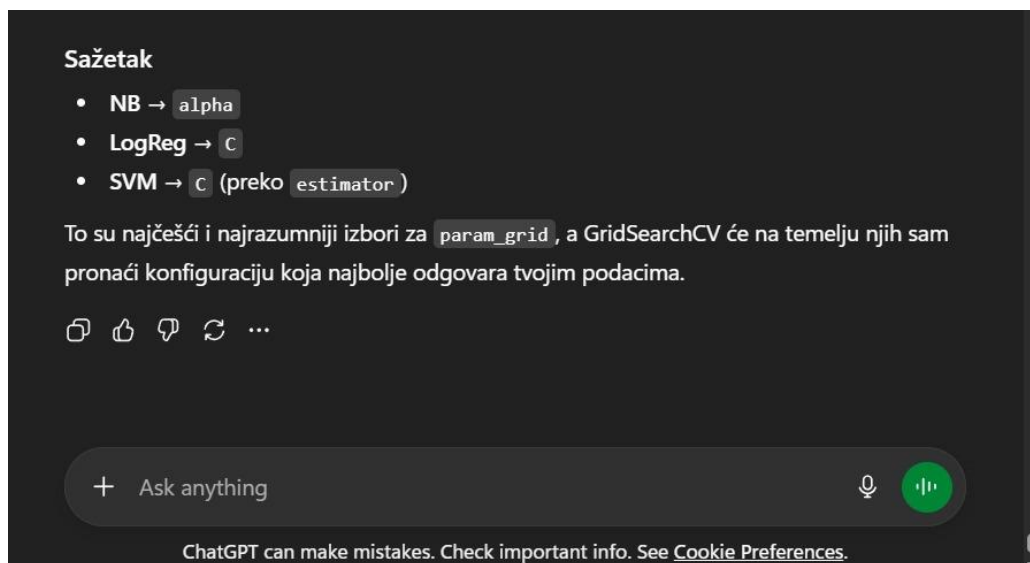
Konverzacija AI alat 2/5 (autorski rad)



Konverzacija AI alat 3/5 (autorski rad)



Konverzacija AI alat 4/5 (autorski rad)



Konverzacija AI alat 5/5 (autorski rad)

3. Razrada teme

U ovom poglavlju detaljno ćemo razraditi problem phishinga i detektiranja takvih poruka pomoću klasičnih metoda strojnog učenja (eng. Machine learning). Na početku ćemo jasnije definirat pojmove phishing i spam-a, a nakon toga opisat metode strojnog učenja koje su korištene u ovom projektu, svaku ukratko prokomentirat i objasniti klasifikaciju poruka. Nakon toga slijedi kritički osvrt, opis implementacije te sam prikaz rada aplikacije i za kraj zaključak teme.

3.1. Phishing i spam

Phishing sam po sebi je vrsta online kriminala koja najčešće zahtjeva korištenje email-a ili u nekim slučajevima SMS-a. Phishing kriminalci se predstavljaju kao tvrtke ili kao druge osobe te pokušavaju od svojih žrtvi zadobiti povjerljive osobne podatke poput OIB-a, podatke o kreditnoj kartici ili nešto treće. Phishing datira još od 1990-tih [8], u vrijeme za koje znamo da sam pojam Internet ili bilo koja riječ čiji je prefiks bio „web“ nije imala neki značaj za ljude u području današnje Hrvatske. No kroz godine kako se sam Internet razvijao, prateći tehnologiju i trendove tako su i ljudi počeli koristiti sve više phishing i online prijevare kako bi profitirali. U današnje vrijeme poznat je termin „spear phishing“, a termin predstavlja postupak kriminalca koji krade javno dostupne osobne podatke koje osobe stavljaju na društvene mreže te koristi se njima u svoju dobrobit [8].

Spam je termin koji se koristi za označavanje neželjenih ili masovno poslanih poruka, najčešće u obliku e-mailova, koje korisnici nisu tražili. Iako "spam" nije akronim za računalnu prijetnju, neki su pokušali dati značenje tom terminu, poput "stupid pointless annoying malware" (glupasti beskorisni dosadni malware) [9]. Postoji mnogo vrsta spam-a, no neki od glavnih su komercijalni spam (poput mail-ova vezani za proizvod neke tvrtke i akcija ili popusta koji se taj dan održavaju), phishing te malware (spam koji sadrži zlonamjerne programe koji mogu naštetiti zdravlju računala) te još mnogi drugi, no vrijedno bi bilo zapamtiti ova tri kao glavne.

3.2. Metode strojnog učenja

Za rješavanje problema detekcije phishing poruka odabran je pristup nadziranog strojnog učenja. Budući da je cilj svaku ulaznu poruku klasificirati u jednu od dvije kategorije (legitimna ili phishing), problem je definiran kao binarna klasifikacija. Tri algoritma koja smo koristili su Multinomial Naive Bayes (NB), Logističku regresiju (LogReg) i Support Vector Machine (SVM) za koju već znamo sa predavanja iz kolegija Uvod u umjetnu inteligenciju.

3.2.1. Multinomial Naive Bayes (NB)

„The Naïve Bayes (NB) classifier is a family of simple probabilistic classifiers based on a common assumption that all features are independent of each other, given the category variable [3].“ Jednostavno rečeno to znači da je svaka riječ neovisna sama za sebe. Nije bitno ako se neka riječ često pojavljuje u phishing porukama, to ne znači da će ona u nekoj drugoj označavati da je ta poruka spam. No iako površan i dosta dobar, ovaj algoritam često zna imati greške te ne klasificira uvijek ispravno. On može često pogrešno klasificirati jer ne uzima u obzir međusobne odnose između tih riječi.

3.2.2. Logistička regresija (LogReg)

Logistička regresija je statistička metoda koja se koristi za predviđanje vjerojatnosti nastanka određenog događaja, kao što je npr. odluka je li e-mail poruka spam ili nije. Za razliku od linearne regresije koja predviđa kontinuirane numeričke vrijednosti, logistička regresija se koristi kada je zavisna varijabla kategorijalna, odnosno kada želimo klasificirati podatke u određene grupe [4]. Osnovni mehanizam ove metode temelji se na transformaciji izlaznih vrijednosti pomoću logističke funkcije. Ova funkcija preslikava bilo koju realnu vrijednost u raspon između 0 i 1, tvoreći pritom karakterističnu krivulju u obliku slova "S". Dobivena vrijednost predstavlja vjerojatnost pripadnosti određenoj klasi [4]. Kako bi se ta vjerojatnost pretvorila u konačnu odluku (binarnu klasifikaciju), definira se prag odlučivanja (*decision threshold*), koji se standardno postavlja na 0.5. Ako je izračunata vjerojatnost veća od tog praga, uzorak se klasificira u jednu klasu (npr. "phishing"), a ako je manja, u drugu (npr. "legit").

3.2.3. Support Vector Machine (SVM)

Metoda potpornih vektora (SVM) je vrlo učinkovit algoritam strojnog učenja koji se koristi za klasifikaciju, osobito za dvoklasnu klasifikaciju, i temelji se na ideji pronalaženja hiperravnine

koja najbolje razdvaja dvije klase podataka. Osnovna ideja SVM-a je pronaći optimalnu hiperravninu koja maksimizira marginu, odnosno udaljenost između dvije klase. SVM koristi potporne vektore, koji su podatkovne točke koje su najbliže hiperravnini i imaju izravan utjecaj na poziciju te hiperravnine [7]. Najbitnije za zapamtiti je da SVM koristi hiperravninu za razdvajanje dviju klasa podataka, maksimizirajući razmak između istih kako bi poboljšao generalizaciju, potporni vektori su točke koje definiraju poziciju hiperravnine, dok kernel funkcije omogućuju rješavanje nelinearnih problema transformiranjem podataka u viši dimenzionalni prostor.

3.3. Kritički osvrt

Implementacija sustava za detekciju *phishing* poruka pokazala je da se primjenom klasičnih algoritama nadziranog strojnog učenja, poput metode potpornih vektora (SVM) i Naivnog Bayesa, mogu postići visoki rezultati točnosti uz minimalne računalne resurse, što ovakvo rješenje čini pogodnim za direktnu integraciju na klijentske uređaje. Ključna prednost razvijenog modela leži u njegovoj brzini i interpretabilnosti, gdje kombinacija leksičkih značajki i inženjeringa strukturnih značajki (poput interpunkcijskih znakova) uspješno prepoznaje standardne obrasce napada, no glavni nedostatak proizlazi iz korištenja TF-IDF vektorizacije koja tekst promatra kao skup nepovezanih riječi, čime se gubi dublji semantički kontekst nužan za prepoznavanje sofisticiranijih napada socijalnog inženjeringa. Iako sustav ne provjerava validnost poveznica u stvarnom vremenu već se oslanja na statičku analizu, on predstavlja iznimno efikasnu "prvu liniju obrane" za filtriranje masovnog zlonamjernog prometa, dok bi se za potpunu robustnost u budućem razvoju preporučio hibridni pristup koji bi uključivao i naprednije jezične modele.

3.4. Opis implementacije

U implementaciji smo napravili sustav kroz četiri koraka: priprema podataka, definiranje pomoćnih klasa za značajke, treniranje modela i na kraju aplikacija (GUI) za isprobavanje modela.

3.4.1. data_prep.py

Ova skripta služi za to da iz više izvora podataka učitamo poruke, očistimo tekst i sve svedemo na isti format. Tekst se "normalizira" (maknu se višestruki razmaci i novi redovi), a labela se mapiraju na dvije klase: phishing i legit. Zatim se uklanjaju prazni i dupli zapisi te se sve sprema u jedan "processed" CSV koji kasnije koristimo za treniranje.

```
data_prep.py > label_u_klasu
1 import argparse
2 import re
3 import pandas as pd
4
5 def procisti_tekst(t: str) -> str:
6     if pd.isna(t):
7         return ""
8     t = str(t)
9     t = t.replace("\r", " ").replace("\n", " ").strip()
10    t = re.sub(r"\s+", " ", t)
11    return t
12
13 def label_u_klasu(lbl) -> str:
14     if pd.isna(lbl):
15         return ""
16     s = str(lbl).strip().lower()
17     if s in {"1", "phishing", "phish", "spam"}:
18         return "phishing"
19     if s in {"0", "legit", "legitimate", "ham", "non-phishing", "not phishing"}:
20         return "legit"
21
22     return s
23
24 def učitaj_spoji(putanja: str, text_cols: list[str], label_col: str) -> pd.DataFrame:
25     df = pd.read_csv(putanja)
26
27     for c in text_cols + [label_col]:
28         if c not in df.columns:
29             raise ValueError(f"U datoteci {putanja} ne postoji stupac: {c}. Postoje: {list(df.columns)}")
30
31     tekst = []
32     for _, row in df.iterrows():
33         dijelovi = []
34         for c in text_cols:
35             dijelovi.append(procisti_tekst(row[c]))
36         tekst.append(" ".join([x for x in dijelovi if x]))
37
38     out = pd.DataFrame({
39         "text": tekst,
40         "label": [label_u_klasu(x) for x in df[label_col]]
41     })
42
43     out = out[(out["text"].str.len() > 0) & (out["label"].str.len() > 0)]
44     out = out.drop_duplicates(subset=["text", "label"]).reset_index(drop=True)
45
46     out = out[out["label"].isin(["phishing", "legit"])].reset_index(drop=True)
47     return out
48
49 def main():
50     parser = argparse.ArgumentParser()
51     parser.add_argument("--out", required=True, help="Izlazni processed CSV")
52     args = parser.parse_args()
```

data_prep.py 1/2 (autorski rad)

```

54
55
56 datasets = [
57     ("data/raw/phishing_email.csv", ["text_combined"], "label"),
58     ("data/raw/Enron.csv", ["subject", "body"], "label"),
59     ("data/raw/Nazario.csv", ["subject", "body"], "label"),
60 ]
61
62 svi = []
63 for put, tcols, lcol in datasets:
64     df = učitaj_spoji(put, tcols, lcol)
65     df["source"] = put.split("/)[-1]
66     svi.append(df)
67
68 spojeno = pd.concat(svi, ignore_index=True)
69
70 spojeno = spojeno.drop_duplicates(subset=["text", "label"]).reset_index(drop=True)
71
72 print("Ukupno zapisa:", len(spojeno))
73 print("Distribucija labela:")
74 print(spojeno["label"].value_counts())
75 print("\nDistribucija po izvoru:")
76 print(spojeno.groupby(["source", "label"]).size())
77
78 spojeno.to_csv(args.out, index=False)
79 print(f"\nSpremio: {args.out}")
80
81 if __name__ == "__main__":
82     main()

```

data_prep.py 2/2 (autorski rad)

3.4.2. pomocne_klase_modela.py

Ovdje smo napravili dio koji pretvara tekst poruke u numeričke značajke koje model može učiti. Koristimo kombinaciju: TF–IDF nad riječima (word n-grami), TF–IDF nad znakovima (char n-grami), te dodatne ručno napravljene značajke poput: duljine poruke, broja URL-ova, broja znamenki, uskličnika, upitnika i velikih slova.

```

pomocne_klase_modela.py > KombiniraniVektorizator > __init__
1 import re
2 import numpy as np
3 from scipy.sparse import csr_matrix, hstack
4 from sklearn.base import BaseEstimator, TransformerMixin
5 from sklearn.feature_extraction.text import TfidfVectorizer
6
7
8 class DodatneZnacajke(BaseEstimator, TransformerMixin):
9     def __init__(self):
10         self.url_re = re.compile(r"(https?://\S+|www\.\S+)", re.IGNORECASE)
11
12     def fit(self, X, y=None):
13         return self
14
15     def transform(self, X):
16         url_broj = []
17         duljina = []
18         broj_brojeva = []
19         broj_uzvicnika = []
20         broj_upitnika = []
21         broj_velikih = []
22
23         for t in X:
24             t = "" if t is None else str(t)
25             duljina.append(len(t))
26             url_broj.append(len(self.url_re.findall(t)))
27             broj_brojeva.append(sum(ch.isdigit() for ch in t))
28             broj_uzvicnika.append(t.count("!"))
29             broj_upitnika.append(t.count("?"))
30             broj_velikih.append(sum(ch.isupper() for ch in t))
31
32         mat = np.array(
33             [duljina, url_broj, broj_brojeva, broj_uzvicnika, broj_upitnika, broj_velikih],
34             dtype=np.float32
35         ).T
36
37         return csr_matrix(mat)
38
39
40 class KombiniraniVektorizator(BaseEstimator, TransformerMixin):
41     def __init__(self, word_ngram=(1, 2), char_ngram=(3, 5), min_df=2, max_df=0.95):
42         self.word_ngram = word_ngram
43         self.char_ngram = char_ngram
44         self.min_df = min_df
45         self.max_df = max_df

```

pomocne_klase_modela.py 1/2 (autorski rad)

```

46
47         self.word = TfidfVectorizer(
48             lowercase=True,
49             ngram_range=self.word_ngram,
50             min_df=self.min_df,
51             max_df=self.max_df
52         )
53         self.char = TfidfVectorizer(
54             lowercase=True,
55             analyzer="char",
56             ngram_range=self.char_ngram,
57             min_df=self.min_df,
58             max_df=self.max_df
59         )
60         self.extra = DodatneZnacajke()
61
62     def fit(self, X, y=None):
63         self.word.fit(X, y)
64         self.char.fit(X, y)
65         self.extra.fit(X, y)
66         return self
67
68     def transform(self, X):
69         Xw = self.word.transform(X)
70         Xc = self.char.transform(X)
71         Xe = self.extra.transform(X)
72         return hstack([Xw, Xc, Xe]).tocsr()

```

pomocne_klase_modela.py 2/2 (autorski rad)

3.4.3. train.py

Ova skripta uzima processed CSV, napravi podjelu na trening i test skup (80/20) i zatim trenira odabrani model. U kodu su podržane tri opcije: Naive Bayes (nb), logistička regresija (logreg) i SVM (svm). Kod SVM-a je napravljena i kalibracija da bi se u GUI-u mogle prikazivati vjerojatnosti (*predict_proba*). Nakon treniranja ispiše se confusion matrix i classification report, izvještaj se spremi u reports/, a cijeli pipeline (značajke + model) se sprema kao .joblib za kasnije korištenje u aplikaciji.

```
train.py > main
1 import argparse
2 import os
3 import joblib
4 import numpy as np
5 import pandas as pd
6
7 from sklearn.model_selection import train_test_split, GridSearchCV
8 from sklearn.pipeline import Pipeline
9 from sklearn.metrics import classification_report, confusion_matrix
10
11 from sklearn.naive_bayes import MultinomialNB
12 from sklearn.linear_model import LogisticRegression
13 from sklearn.svm import LinearSVC
14 from sklearn.calibration import CalibratedClassifierCV
15 from pomocne_klase_modela import KombiniraniVektorizator
16
17
18 def učitaj_processed(putanja: str) -> pd.DataFrame:
19     df = pd.read_csv(putanja)
20     df = df[["text", "label"]].dropna()
21     df["text"] = df["text"].astype(str)
22     df["label"] = df["label"].astype(str)
23     df = df.drop_duplicates(subset=["text", "label"])
24     return df
25
26
27 def napravi_model(naziv: str):
28     naziv = naziv.lower()
29
30     if naziv == "nb":
31         return MultinomialNB()
32
33     if naziv == "logreg":
34         return LogisticRegression(max_iter=3000, class_weight="balanced")
35
36     if naziv == "svm":
37         baza = LinearSVC(max_iter=5000)
38         return CalibratedClassifierCV(baza, method="sigmoid", cv=3)
39
40     raise ValueError("Model mora biti: nb, logreg ili svm")
41
42
43 def main():
44     parser = argparse.ArgumentParser()
45     parser.add_argument("--data", required=True, help="data/processed/processed.csv")
46     parser.add_argument("--model", default="svm", choices=["nb", "logreg", "svm"])
47     parser.add_argument("--out", default="models/phishing_model.joblib")
48     parser.add_argument("--report_dir", default="reports")
49     parser.add_argument("--tuning", action="store_true", help="Pokreni GridSearchCV (sporije, ali bolje)")
50     args = parser.parse_args()
51
52     os.makedirs(os.path.dirname(args.out), exist_ok=True)
53     os.makedirs(args.report_dir, exist_ok=True)
54
55     df = učitaj_processed(args.data)
56     X = df["text"].values
57     y = df["label"].values
```

train.py 1/2 (autorski rad)

```

58 X_train, X_test, y_train, y_test = train_test_split(
59     X, y, test_size=0.2, random_state=42, stratify=y
60 )
61
62 vekt = KombiraniVektorizator()
63 clf = napravi_model(args.model)
64
65 pipeline = Pipeline([
66     ("feat", vekt),
67     ("clf", clf)
68 ])
69
70
71 if args.tuning:
72     param_grid = {
73         "feat__min_df": [1, 2, 3],
74         "feat__max_df": [0.90, 0.95, 0.98],
75         "feat__word_ngram": [(1, 1), (1, 2)],
76         "feat__char_ngram": [(3, 5), (4, 6)],
77     }
78
79     if args.model == "nb":
80         param_grid["clf__alpha"] = [0.1, 0.5, 1.0]
81     elif args.model == "logreg":
82         param_grid["clf__C"] = [0.5, 1.0, 2.0, 4.0]
83     elif args.model == "svm":
84         param_grid["clf__estimator__C"] = [0.5, 1.0, 2.0, 4.0]
85
86     gs = GridSearchCV(
87         pipeline,
88         param_grid=param_grid,
89         scoring="f1_macro",
90         cv=4,
91         n_jobs=-1,
92         verbose=2
93     )
94     gs.fit(X_train, y_train)
95     pipeline = gs.best_estimator_
96     print("Najbolji parametri:", gs.best_params_)
97 else:
98     pipeline.fit(X_train, y_train)
99
100 y_pred = pipeline.predict(X_test)
101
102 cm = confusion_matrix(y_test, y_pred, labels=np.unique(y))
103 rep = classification_report(y_test, y_pred, digits=4)
104
105 print("Confusion matrix:\n", cm)
106 print("\nClassification report:\n", rep)
107
108 with open(os.path.join(args.report_dir, "classification_report.txt"), "w", encoding="utf-8") as f:
109     f.write(rep)
110
111 joblib.dump(pipeline, args.out)
112 print(f"\nModel spremljen u: {args.out}")
113 print(f"Izvjestaj u: {args.report_dir}/classification_report.txt")
114
115
116 if __name__ == "__main__":
117     main()

```

train.py 2/2 (autorski rad)

3.4.4. gui_tkinter.py

Ovo je jednostavna desktop aplikacija napravljena u Tkinteru. Ona učitava spremljeni model (phishing_model.joblib) i omogućuje korisniku da zalijepi tekst poruke i klikne "Provjeri". Aplikacija zatim ispiše rezultat PHISHING ili LEGIT, a ako model podržava vjerojatnosti, prikazuje i kolika je vjerojatnost za svaku klasu. Time se model može praktično testirati bez pokretanja skripti kroz terminal.

```

gui_tkinter.py > PhishingApp > __init__
1 from pomocne_klase_modela import KombiniraniVektorizator, DodatneZnacajke
2 import tkinter as tk
3 from tkinter import messagebox, scrolledtext
4 import os
5 import joblib
6
7
8 ROOT_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
9 MODEL_PATH = os.path.join(ROOT_DIR, "models", "phishing_model.joblib")
10
11
12 class PhishingApp(tk.Tk):
13     def __init__(self):
14         super().__init__()
15
16         self.title("Detekcija phishing poruka (LUI projekt)")
17         self.geometry("800x520")
18
19         if not os.path.exists(MODEL_PATH):
20             messagebox.showerror(
21                 "Greška",
22                 f"Model nije pronađen: {MODEL_PATH}\n\nPrvo pokreni train.py da se model spremi."
23             )
24             self.destroy()
25             return
26
27         try:
28             self.model = joblib.load(MODEL_PATH)
29         except Exception as e:
30             messagebox.showerror("Greška", f"Ne mogu učitati model.\n\nDetalji: {e}")
31             self.destroy()
32             return
33
34         self._napravi_ui()
35
36     def _napravi_ui(self):
37         naslov = tk.Label(self, text="Detekcija phishing poruka (email/SMS) - klasični ML", font=("Arial", 14, "bold"))
38         naslov.pack(pady=10)
39
40         opis = tk.Label(
41             self,
42             text="Unesi tekst poruke i klikni 'Provjeri'.",
43             font=("Arial", 11)
44         )
45         opis.pack(pady=5)
46
47         okvir = tk.Frame(self)
48         okvir.pack(fill="both", expand=True, padx=12, pady=10)
49
50         self.ulaz = scrolledtext.ScrolledText(okvir, wrap=tk.WORD, font=("Arial", 11))
51         self.ulaz.pack(fill="both", expand=True)
52
53         gumbi = tk.Frame(self)
54         gumbi.pack(pady=10)
55
56         btn_provjeri = tk.Button(gumbi, text="Provjeri", width=15, command=self.provjeri)
57         btn_provjeri.grid(row=0, column=0, padx=8)
58
59         btn_ocisti = tk.Button(gumbi, text="Očisti", width=15, command=self.ocisti)
60         btn_ocisti.grid(row=0, column=1, padx=8)
61
62         self.rezultat = tk.Label(self, text="Rezultat: -", font=("Arial", 12, "bold"))
63         self.rezultat.pack(pady=8)
64
65         self.detalji = tk.Label(self, text="", font=("Arial", 11))
66         self.detalji.pack(pady=2)
67

```

gui_tkinter.py 1/2 (autorski rad)

```

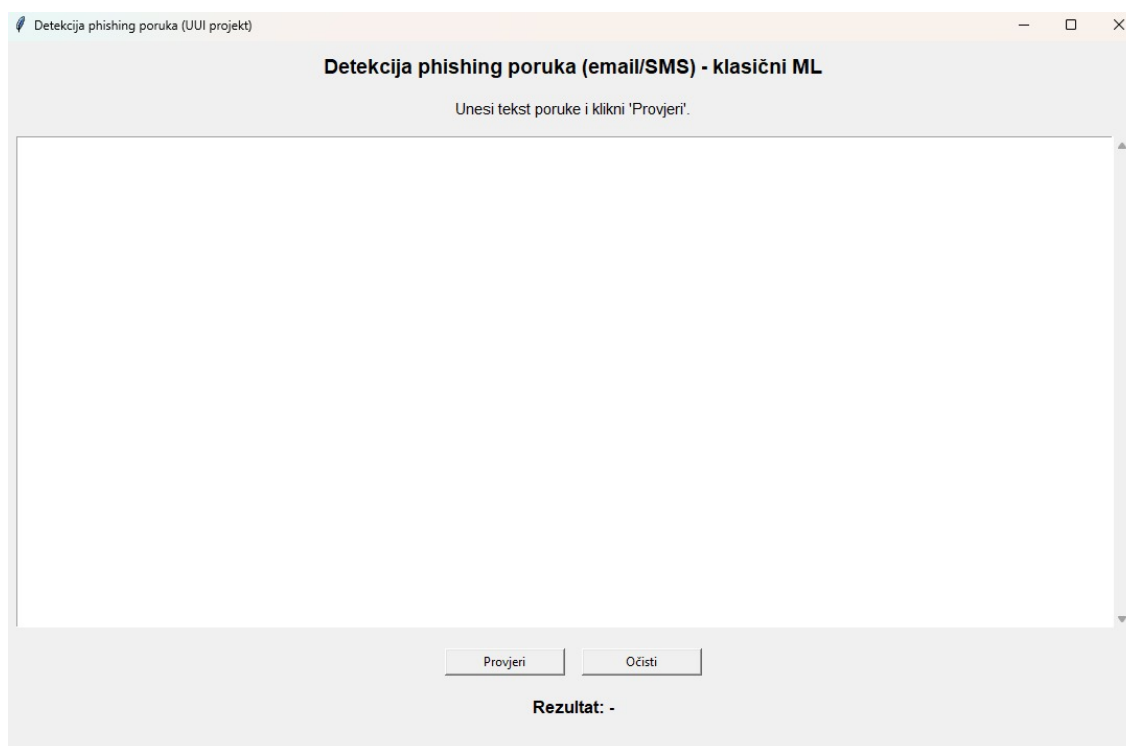
69     def ocisti(self):
70         self.ulaz.delete("1.0", tk.END)
71         self.rezultat.config(text="Rezultat: -")
72         self.detalji.config(text="")
73
74     def provjeri(self):
75         tekst = self.ulaz.get("1.0", tk.END).strip()
76         if not tekst:
77             messagebox.showwarning("Upozorenje", "Unesi tekst poruke.")
78             return
79
80         try:
81             pred = self.model.predict([tekst])[0]
82         except Exception as e:
83             messagebox.showerror("Greška", f"Ne mogu napraviti predikciju.\n\nDetalji:\n{e}")
84             return
85
86         if pred == "phishing":
87             self.rezultat.config(text="Rezultat: PHISHING", fg="red")
88         else:
89             self.rezultat.config(text="Rezultat: LEGIT", fg="green")
90
91         if hasattr(self.model, "predict_proba"):
92             proba = self.model.predict_proba([tekst])[0]
93             klase = list(self.model.classes_)
94             score = {k: float(p) for k, p in zip(klase, proba)}
95
96             p_phish = score.get("phishing", 0.0)
97             p_legit = score.get("legit", 0.0)
98
99             self.detalji.config(text=f"Vjerojatnost phishing: {p_phish:.4f} | legit: {p_legit:.4f}")
100         else:
101             self.detalji.config(text="(Model ne vraća vjerojatnosti.)")
102
103
104 if __name__ == "__main__":
105     app = PhishingApp()
106     app.mainloop()

```

gui_tkinter.py 2/2 (autorski rad)

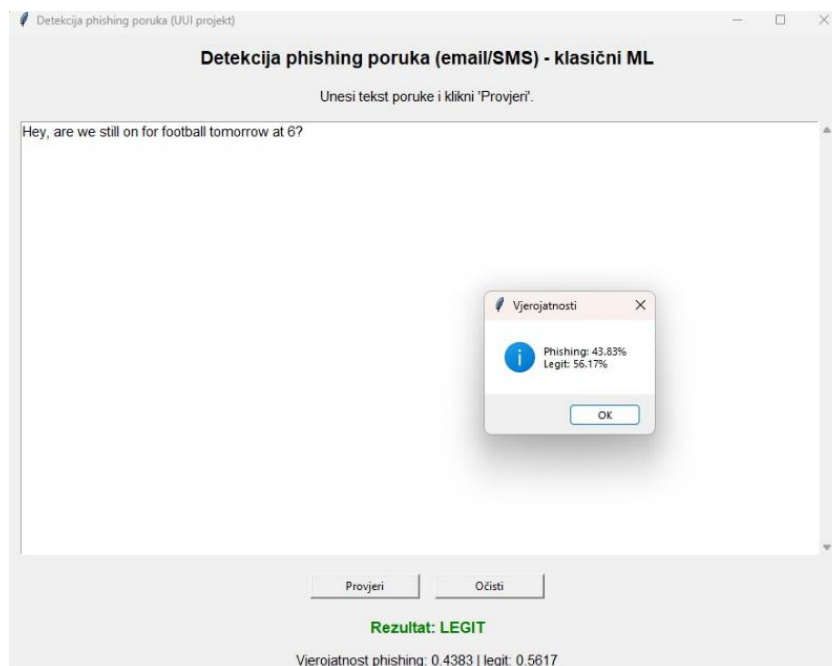
3.5. Prikaz rada aplikacije

Rad naše aplikacije je jednostavan te krajnjem korisniku bi trebao biti poprilično intuitivan. Kao što se može vidjeti na fotografiji priloženoj ispod aplikacija se otvara u novom prozoru sa naslovom programa, te uputama odmah ispod. Poruku koju želite provjeriti za spam unosite u prazno bijelo tekstualno polje te ukoliko želite provjeriti taj tekst, klikom na donji gumb „Provjeri“ program će vam ispisati je li poruka PHISHING ili LEGIT, te će poruka biti obojana u pripadajuće boje da ne dođe do zabune. Crvena boja i tekst PHISHING označavaju da poruka nije sigurna za korisnika te da se kriju zlonamjerni motivi, no ukoliko se pojavi zeleno obojan tekst LEGIT to označava da je program procijenio da je poruka sigurna za krajnjeg korisnika. Također prilikom dobivanja rezultata program korisniku ispisuje kolike su vjerojatnosti da je određena poruka phishing ili legit. A ukoliko se zabunite prilikom početnog unosa teksta, dolje niže se nalazi gumb „Očisti“ kojim brišete sve unesene znakove.

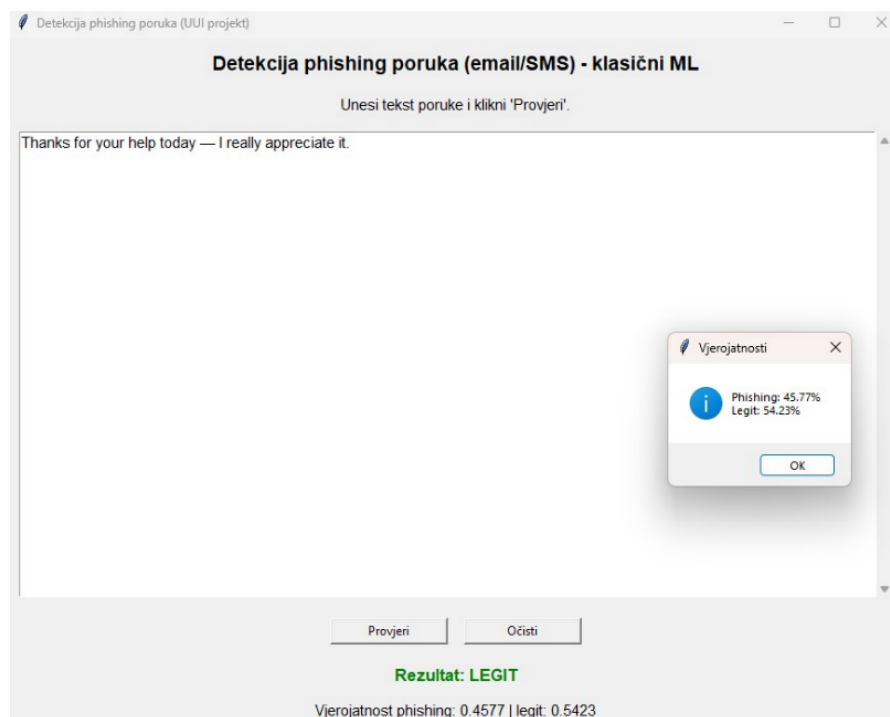


Sučelje aplikacije (autorski rad)

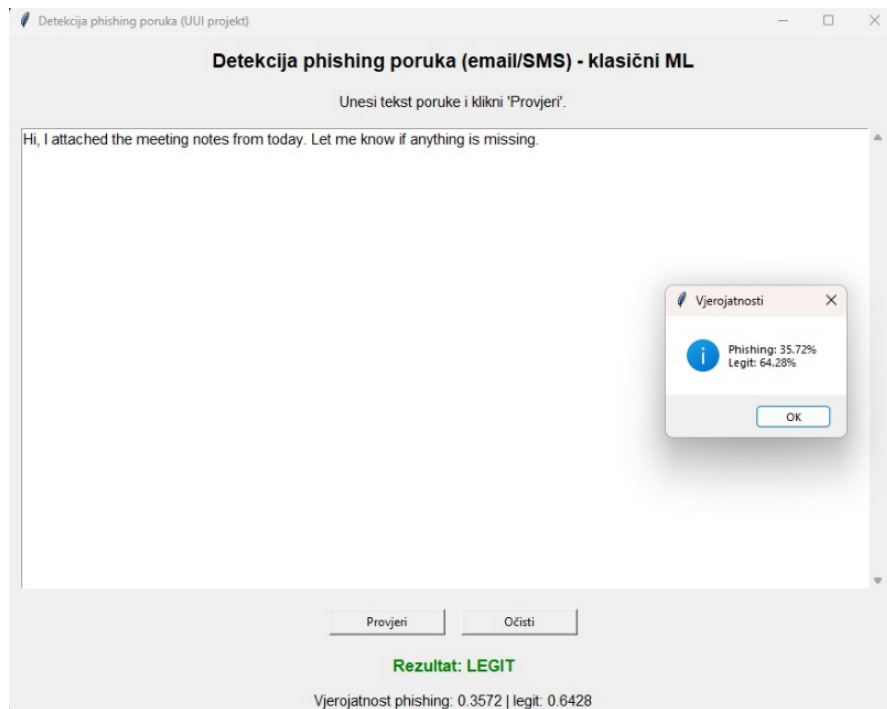
A sada nekoliko primjera dobronamjernih (LEGIT) i zlonamjernih (PHISHING) poruka:



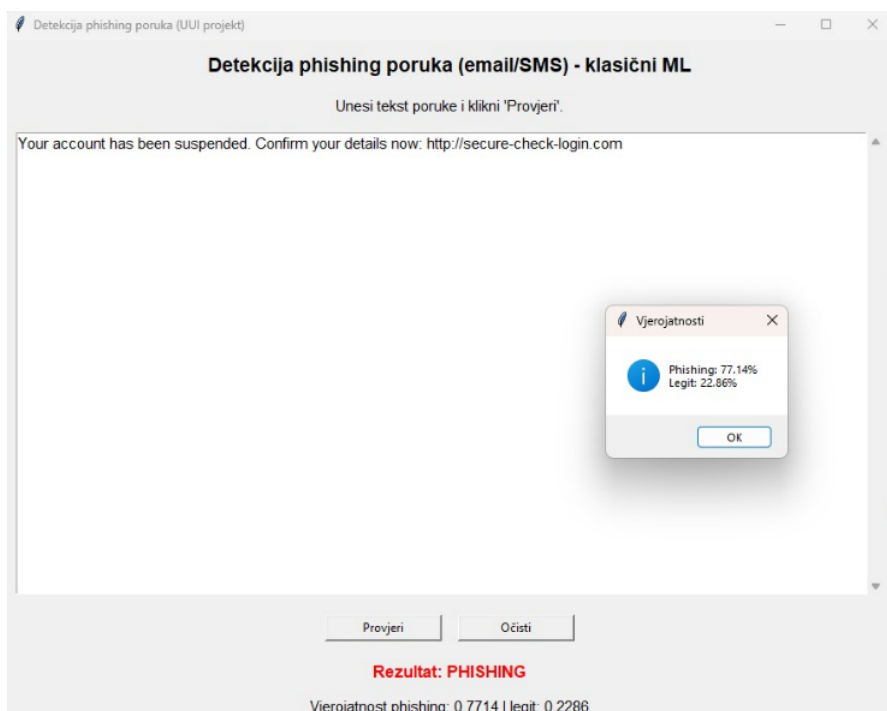
Legit 1 (autorski rad)



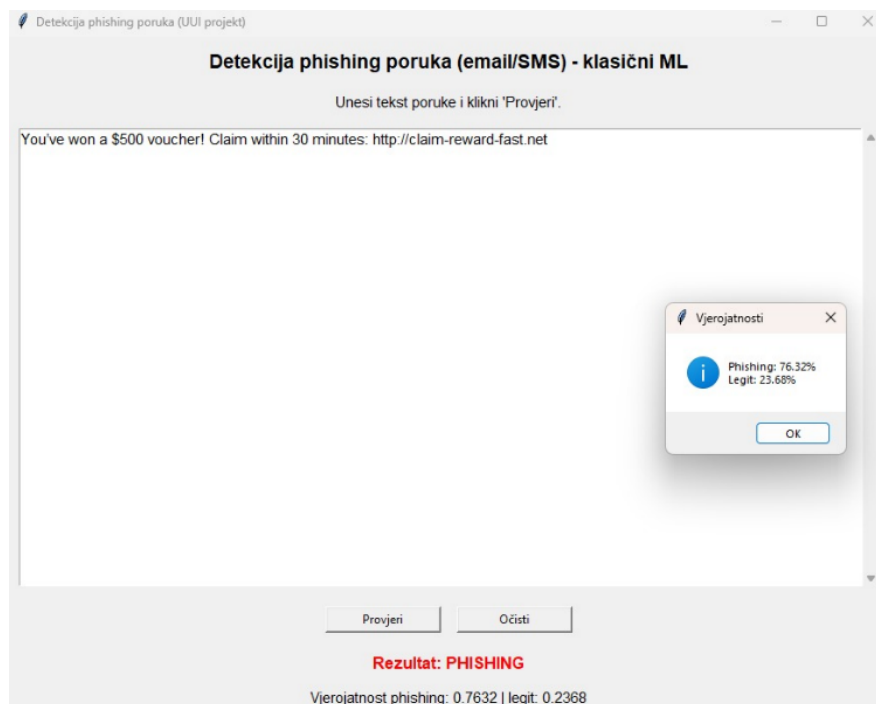
Legit 2 (autorski rad)



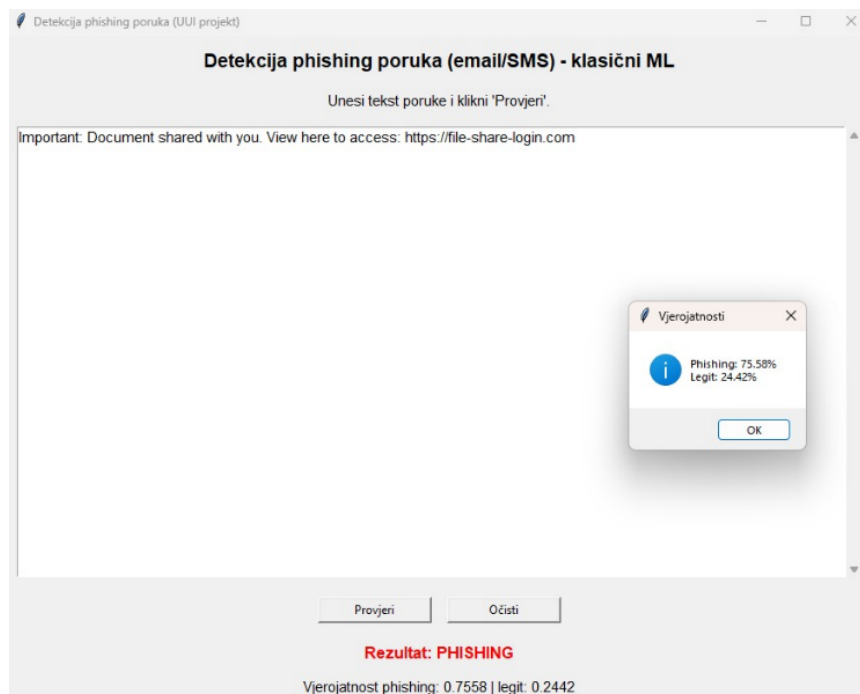
Legit 3 (autorski rad)



Phishing 1 (autorski rad)



Phishing 2 (autorski rad)



Phishing 3 (autorski rad)

4. Zaključak

U ovom radu razvijen je sustav za detekciju phishing poruka u e-mail i SMS komunikaciji korištenjem klasičnih metoda strojnog učenja. Cilj rada bio je ispitati može li se analizom sadržaja poruka razlikovati phishing od legitimnih poruka uz prihvatljivu točnost i bez korištenja složenih metoda dubokog učenja. U razradi rada korištene su metode nadziranog učenja, konkretno Multinomial Naive Bayes, logistička regresija i Support Vector Machine (SVM). Modeli su trenirani i evaluirani korištenjem standardnih metrika, a razvijena je i jednostavna Tkinter aplikacija za praktičnu provjeru poruka. Kao alati korišteni su Python, Visual Studio Code za izradu aplikacije i LaTeX za izradu dokumentacije. Rezultati rada potvrđuju početnu pretpostavku da klasične metode strojnog učenja mogu učinkovito prepoznavati phishing poruke te predstavljaju dobar kompromis između brzine, jednostavnosti i točnosti. Ipak, pokazalo se da analiza samog teksta ima ograničenja u složenijim slučajevima, što upućuje na mogućnost budućih poboljšanja dodavanjem novih značajki ili dodatnih izvora informacija.

Popis literature

- [1] I. Fette, N. Sadeh, and A. Tomasic, (2007.), "Learning to detect phishing emails," in *Proc. 16th Int. Conf. World Wide Web (WWW '07)*, Banff, AB, Canada, pp. 649–656
- [2] H. Faris, A. M. Al-Zoubi, A. A. Heidari, I. Aljarah, M. Mafarja, M. A. Hassonah, and H. Fujita, (2019.), "An intelligent system for spam detection and identification of the most relevant features based on evolutionary Random Weight Networks," *Information Fusion*, vol. 48, pp. 67–83,
- [3] Xu, S. (2016). Bayesian Naïve Bayes classifiers to text classification. *Journal of Information Science*, 44(1), 48-59.
- [4] Lee F. („n.d.“) What is logistic regression? Preuzeto 02.01.2026 s <https://www.ibm.com/think/topics/logistic-regression>
- [5] Kavlakoglu E. („n.d.“) What are Naïve Bayes classifiers? Preuzeto 02.01.2026 s <https://www.ibm.com/think/topics/naive-bayes>
- [6] Kavlakoglu E. („n.d.“) What are support vector machines (SVMs)? Preuzeto 02.01.2026 s <https://www.ibm.com/think/topics/support-vector-machine>
- [7] Oreški D. (2026). Nadzirano učenje. Multimedijski sustavi [Moodle]. Sveučilište u Zagrebu, Fakultet organizacije i informatike, Varaždin
- [8] The History of Phishing Attacks (2023). Preuzeto 03.01.2026. s <https://cofense.com/knowledge-center/history-of-phishing/>
- [9] SPAM What is computer scam? „n.d.“ Preuzeto 05.1.2026. s <https://www.malwarebytes.com/spam>

Popis slika

Sve slike su autorski rad.